

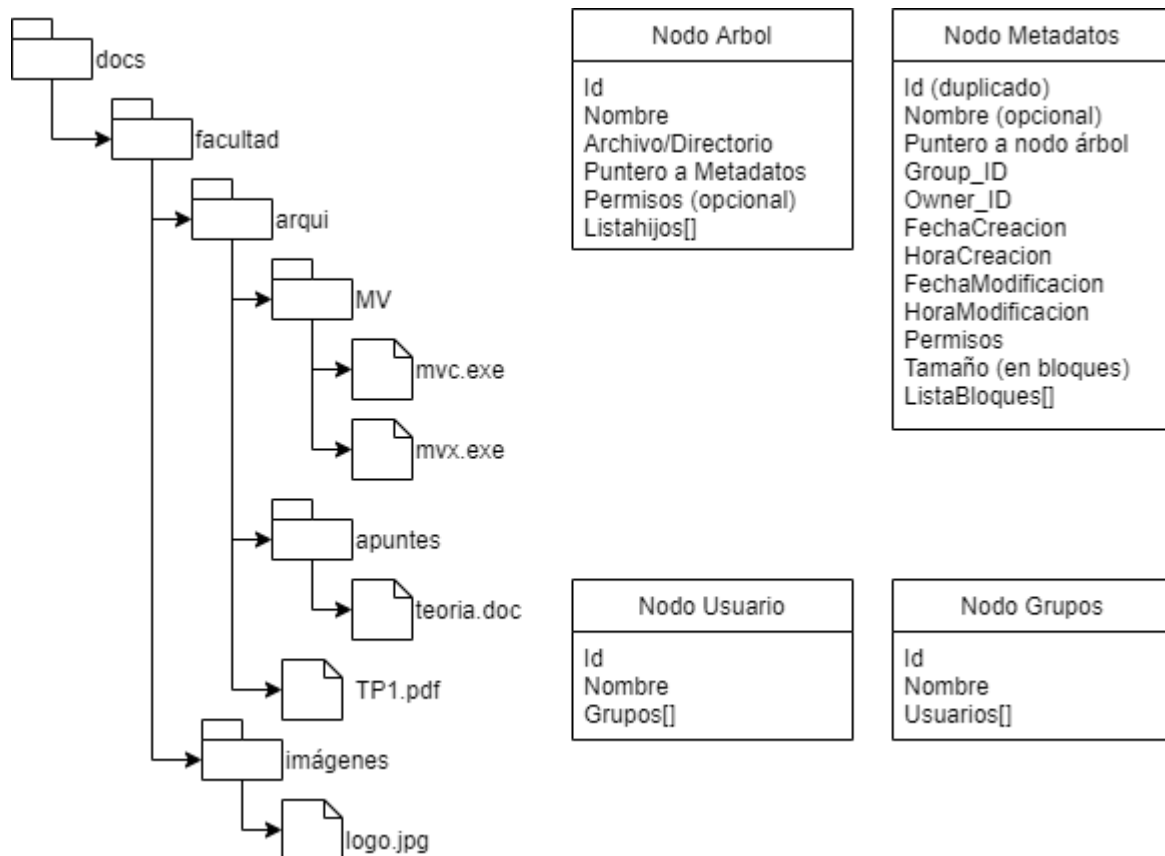
Visión general

Un sistema de archivos es representado por un árbol. Cada nodo del árbol puede ser un archivo o un directorio (carpeta). Los nodos directorios pueden tener hijos que pueden ser archivos o directorios.

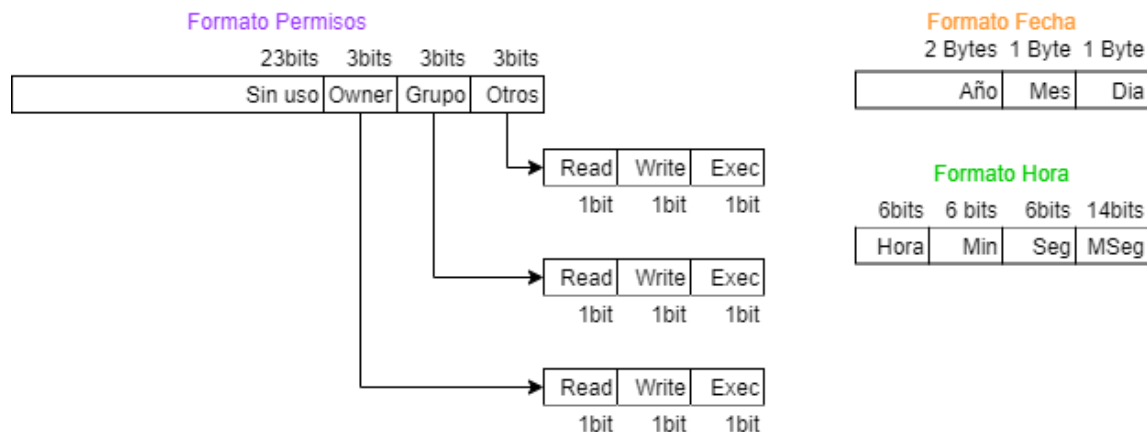
Cada nodo del árbol a su vez tiene una estructura con los metadatos del archivo/directorio (Nombre, fecha de creación, fecha de modificación , tamaño, etc).

Cada archivo o directorio está asociado a un usuario llamado **owner** (el dueño del archivo) y a un grupo (**group**) al que pertenece el archivo.

Además el sistema de archivos tiene una lista de usuarios, donde cada usuario pertenece a 1 o más grupos. Y una lista de grupos donde cada grupo contiene a 1 o más usuarios. La información es coherente y simétrica, si un usuario pertenece a un grupo, ese grupo contendrá al usuario. No puede haber 2 usuarios con el mismo nombres, ni 2 grupos con el mismo nombre.



Tipos de Datos



Formato de fecha: 1 Celda de 4 bytes, en los cuales son, de más significativos a menos significativos: 2 bytes para el año, 1 byte para el mes (1 a 12) y 1 byte para el día (1 a 31).

Formato de hora: 1 celda de 32 bits, en los cuales son, de más significativo a menos significativos: 6 bits para la hora (0 a 23), 6 bits para los minutos (de 0 a 59), 6 bits los segundos (0 a 59) y 14 bits para milésimas de segundos (0 a 999).

Formato de Permisos:

Representan el acceso que puede tener un usuario a un archivo o directorio. Es un atributo propio del archivo o directorio (carpeta). Consta de 1 celda donde solo se utilizan los 9 bits menos significativos.

De más significativo a menos significativo son: 3 bits para el **owner**, 3 para **grupo** y 3 bits para **otros** usuarios.

A su vez cada permiso se subdivide en 1 bit para acceso de lectura, 1 bit para acceso escritura y 1 bit para acceso de ejecución.

Cuando el bit tiene valor 1 significa que el permiso está otorgado, cuando está en 0 el permiso está denegado.

Se recomienda el uso de sistema de numeración octal para trabajar con permisos.

Ejemplos:

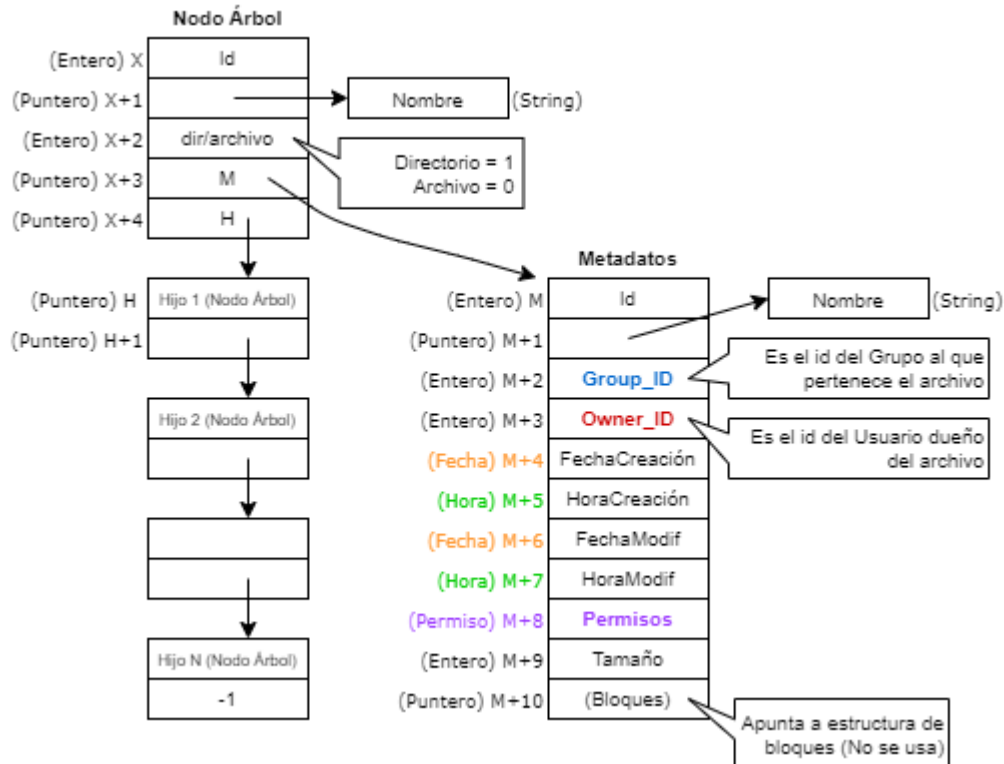
- Si en permisos hay un **@777** (todos los bits en 1) quiere decir que tanto el owner, como el miembro del grupo, como cualquier usuario tiene permiso de lectura escritura y ejecución.
- Si en permisos hay un **@751** (binario 111 101 001) quiere decir que el owner puede leer, escribir y ejecutar; el miembro del grupo puede leer y ejecutar pero no escribir; y por último el resto de los usuarios pueden solo ejecutar.

Para validar si un usuario tiene permisos se debe seguir el siguiente criterio:

- 1) Verificar si es el owner del archivo, y luego mirar solo los permisos del owner (**@700**)
- 2) Verificar si es miembro del grupo, y luego mirar solo los permisos del grupo (**@070**)
- 3) Si no entró ninguno de los casos anterior, mirar los permisos de otros (**@007**)

Estructuras

Árbol



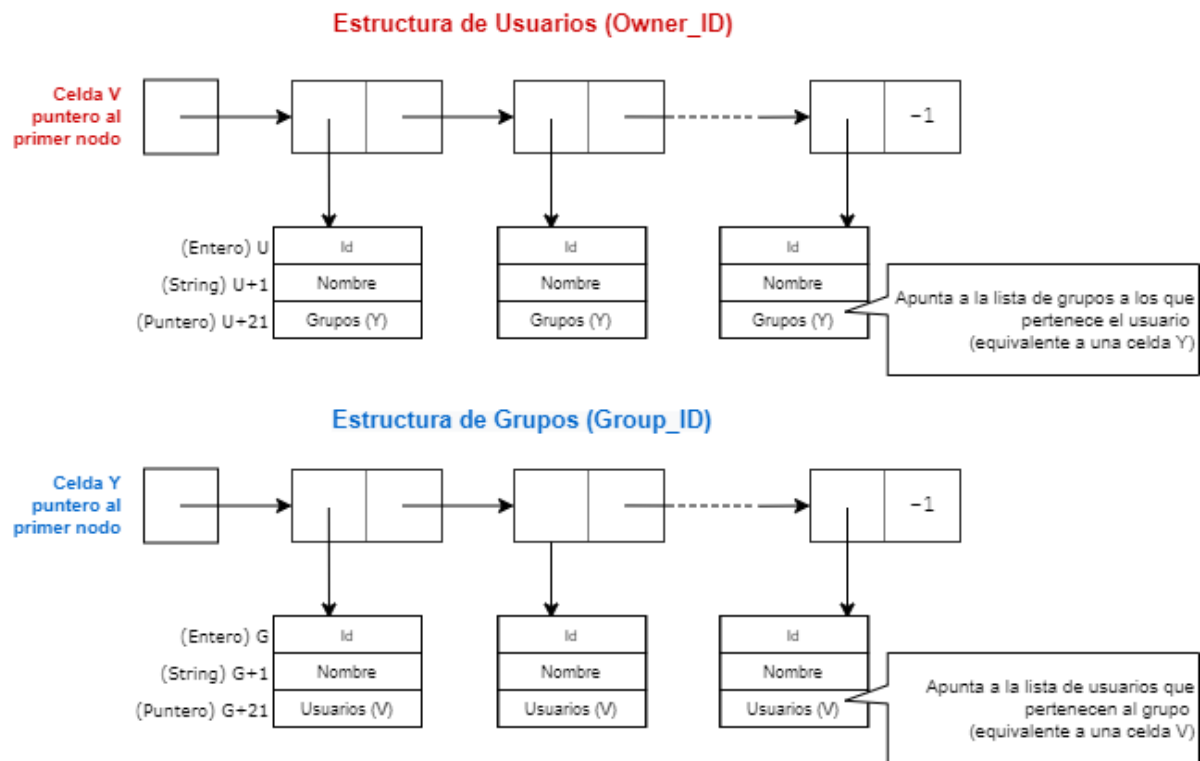
Los archivos y directorios (carpetas) se representan con un nodo de un árbol en una dirección X de memoria. Se diferencia archivos de directorios por el campo en X+2 que si tiene un **1** el nodo corresponde a un **directorio** y si tiene un **0** corresponde a un **archivo**.

Cada nodo del árbol apunta en X+3 a una estructura de metadatos de los archivos/directorios que comienza en una celda M. En los metadatos del archivo se encuentra el **id del usuario** (entero positivo) que es dueño del archivo (OWNER_ID) en M+3 y el **id del grupo** (entero positivo) al que pertenece el archivo (GROUP_ID) en M+2.

Los campos tamaño (M+9) y bloques (M+10) **solo son válidos para los archivos**.

Si el nodo **es un directorio**, en X+4 **apunta a una lista de hijos**. Cada nodo de la lista tiene un puntero a nodo de árbol y un puntero al siguiente. **Si** el directorio **está vacío** en X+4 habrá un -1 (null).

Usuarios y grupos



La estructura de **usuarios** consta de **1 celda V que apunta a una lista** dinámica enlazada **con sublistas**. Las sublistas de grupos (Y) contienen la lista de grupos a los que pertenece el usuario siguiendo la estructura de grupos de la imagen.

La estructura de **grupos**, al igual que la de usuario, consta de **1 celda Y que apunta a una lista** dinámica enlazada **con sublistas**. Las sublistas de usuarios (V) contienen la lista de usuarios que pertenecen al grupo siguiendo la estructura de usuarios de la imagen.

Tanto grupos como usuarios tienen idénticas estructuras, solo cambia el contenido.

Ejercicio

Escriba la (o las) subrutina(s) necesaria(s) para recibir un puntero a un directorio (X) raíz del sistema de archivos, un puntero a un string para la ruta de un directorio, la dirección de memoria (Y) con el puntero a la estructura de grupos del sistema y el id del usuario que realiza la acción. La subrutina debe **encontrar el directorio desde la raíz y listar los nombres de los archivos con las acciones que puede realizar el usuario sobre los mismos**. Debe verificar que permisos tiene el usuario sobre cada archivo indicando rwx si tiene todos los permisos o reemplazar cada letra con “-” si no puede realizar alguna de las acciones, por ejemplo si solo puede leer indicaría “r--”. Si no encuentra el directorio no escribe nada.

NOTA: El string con nombre y ruta comienza con ‘/’ y se utiliza el mismo símbolo para separar entre directorios, el último tramo es el nombre del archivo.

Ejemplo ruta: /docs/facultad/arqui/MV

Ejemplo resultado:

rwx mvc.exe

r-x mvx.exe

La subrutina principal debe ser equivalente al siguiente protocolo en C:

```
void list (TArbol* raiz, String ruta, TGruposs* grupos, int idUsuario)
```

;LIST: Busca e imprime toda la lista de archivos con los permisos del usuario.

```
    push    <idUsuario>
    push    <Celda Y grupos>
    push    <ruta>
    push    <Celda X raíz del árbol>
    call    LIST
    add     sp, 4
```

;PRINT_FILES: Imprime toda la lista de archivos con los permisos del usuario.

```
    push    <idUsuario>
    push    <Celda Y grupos>
    push    <Celda X del dir a imprimir>
    call    PRINT_FILES
    add     sp, 3
```

;GET_PERMISOS: devuelve en ax los permisos del usuario

```
    push    <idUsuario>
    push    <Celda Y grupos>
    push    <metadatos celda M>
    call    GET_PERMISOS
    add     sp, 3 ;en AX devuelve en los últimos 3 bits
```

```
;los permisos del usuario sobre el
;archivo
```

```
ENDSTR      equ    0
null        equ    -1
```

```
;-----
;-----
```

```
;Busca el directorio y luego llama a la impresión.
```

```
LIST:        push    bp
              mov     bp, sp
              push    ax
              push    bx
              push    dx
              push    ex
              push    fx
```

```
              mov     bx, [bp+3]
              add     bx, 1
              mov     dx, bx
otroC:        cmp     dx, '/'
              jz      dirintermedio
              cmp     dx, ENDSTR
              jz      dirfinal
              add     dx, 1
              jmp     otroC
```

```
;
; /docs/facultad/arqui/MV
; ^bx ^dx
dirintermedio: mov     fx, 0
               mov     [dx], ENDSTR
               jmp     buscaH
```

```
;
; /docs/facultad/arqui/MV
;          bx^ ^dx
dirfinal:     mov     fx, 1
               jmp     buscaH
```

```
;ciclo para buscar el hijo del dir que tiene el nombre buscado.
```

```
;
; /docs/facultad/arqui/MV
; ^bx ^dx
```

```

;[bx] apunta al comienzo del nombre
;[dx] apunta al final (donde puso \0)
buscaH:      mov     ax, [bp+2]
             add     ax, 4
buscaD:      cmp     [ax], null
             jz      finLIST ;se va sin hacer nada
             mov     ax, [ax]
             mov     ex, [ax]
             mov     ex, [ex+1]
             scmp    [bx], [ex]
             jz      encontroD
             add     ax, 1
             jmp     buscaD

;quiere decir que encontró el nombre del directorio.
encontroD:   cmp     fx, 1
             jz      invocaPrint
             mov     [dx], '/' ;vuelve a ubicar el divisor
;
; /docs/facultad/arqui/MV
; ^bx ^dx
             push    [bp+5] ;<idUsuario>
             push    [bp+4] ;<Celda Y grupos>
             push    dx    ;<ruta>
             push    [ax]  ;<Celda X del árbol>
             call    LIST
             add     sp, 4
             jmp     finLIST

;
; /docs/facultad/arqui/MV
;          bx^ ^dx
invocaPrint:  push    [bp+5] ;<idUsuario>
             push    [bp+4] ;<Celda Y grupos>
             push    [ax]  ;<Celda X del dir a imprimir>
             call    PRINT_FILES
             add     sp, 3
             jmp     finLIST

finLIST:     pop     fx
             pop     ex
             pop     dx
             pop     bx
             pop     ax
             mov     sp, bp
             pop     bp
             ret

```

```

;-----
;-----
;Imprime toda la lista de archivos con los permisos del usuario.
PRINT_FILES:    push        bp
                mov         bp, sp
                sub         sp, 1          ;reservo variable local
                push        ax
                push        bx
                push        cx
                push        dx
                push        ex
                push        fx

                mov         bx, [bp+2] ; Celda X del dir a imprimir
                mov         bx, [bx+4] ; en bx queda *nodo lista hijos

otrof:          cmp         bx, null
                jz          finPRINT
                mov         ex, [bx]
                mov         ex, [ex+3] ; en ex *metadatos (M)

; escribe rwx o --- según permisos
                push        [bp+4] ;<idUsuario>
                push        [bp+3] ;<Celda Y grupos>
                push        dx      ;metadatos
                call        GET_PERMISOS
                add         sp, 3 ;en AX devuelve en los últimos 3 bits
                                ;los permisos del usuario sobre el
                                ;archivo

                mov         [bp-1], ax ; uso variable local
                mov         cx, 1
                sys         %5        ;en dx queda la celda disponible
                cmp         dx, null
                jz          finPRINT ;se va por falta de memoria.
                xor         ax, ax
                or          ax, %800 ;omite promt
                or          ax, %100 ;omite endlne
                or          ax, %010 ;formato caracter
                ; mov         ax, %910

; vemos si lee r
                mov         fx, [bp-1]
                and         fx, %4 ;b0100 - read

```



```

                                jz         nolee
                                mov         [dx], 'r'
                                sys         %2
                                jmp         esc
nolee:                          mov         [dx], '-'
                                sys         %2

;vemos si escribe
esc:                             mov         fx, [bp-1]
                                and         fx, %2 ;b0010 - write
                                jz         noescribe
                                mov         [dx], 'w'
                                sys         %2
                                jmp         exec
noescribe:                       move        [dx], '-'
                                sys         %2

;vemos si ejecuta
esc:                             mov         fx, [bp-1]
                                and         fx, %1 ;b0001 - exec
                                jz         noexec
                                mov         [dx], 'x'
                                sys         %2
                                jmp         espacio
noexec:                          move        [dx], '-'
                                sys         %2

espacio:                         mov         [dx], ' '
                                sys         %2
                                sys         %6 ;libera memoria

; escribe el nombre del archivo
                                mov         ax, %800
                                mov         dx, [ex+1]
                                sys         %4

                                mov         bx, [bx+1] ;siguiente hijo
                                jmp         otrof

finPRINT:                        pop         fx
                                pop         ex
                                pop         dx
                                pop         cx
                                pop         bx
                                pop         ax
                                mov         sp, bp

```

```

                pop        bp
                ret

;-----
;-----
; devuelve en ax los permisos del usuario
GET_PERMISOS:   push      bp
                mov       bp, sp
                push      bx
                push      dx
                push      ex

                mov       ex, [bp+2] ;celda M metadatos
                mov       dx, [bp+3] ;celda Y grupos
                        ;[bp+4] ;id usuario

                mov       ax, [ex+8] ;todos los permisos

                cmp       [ex+3], [bp+4]
                jz        esowner

;busca el grupo
buscaG:         mov       dx, [dx]
                cmp       dx, null
                jz        singrupa
                mov       bx,[dx]
                cmp       [bx],[ex+2] ; compara id grupo
                jz        encontroG
                mov       dx, [dx+1]
                jmp        buscaG

;con el grupo, busca si el usuario está en el grupo
encontroG:     mov       dx, [bx+21] ;apunta a una celda Y
                mov       dx, [dx]
buscaU:        cmp       dx, null
                jz        singrupa ;hace lo mismo que si
                        ;no hubiera encontrado grupo

                mov       bx,[dx]
                cmp       [bx],[bp+4] ; compara id usuario
                jz        esgrupo
                mov       dx, [dx+1]
                jmp        buscaU

singrupa:      jmp        esotro

```

;según los permisos que le toca ver, es lo que devuelve en Ax

esowner: shr ax, 3

esgrupo: shr ax, 3

esotro: and ax, %7 ; b0111

FIN_GET_PERMISOS: pop ex

pop dx

pop bx

mov sp, bp

pop bp

ret